

```
krishnamohan_koyya@instance-1:~$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT (S)	AGE
kafka	NodePort	10.108.141.251	<none>	9092:30092/TCP	38m
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	11h
mysqldb	ClusterIP	None	<none>	3306/TCP	38m
ums-add-service	LoadBalancer	10.96.38.181	10.96.38.181	8080:31248/TCP	38m
zookeeper	NodePort	10.107.137.176	<none>	2181:30181/TCP	38m

Figure 3: Kubernetes services

```
krishnamohan_koyya@instance-1:~$ kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
kafka	1/1	1	1	39m
mysqldb	1/1	1	1	39m
ums-add-service	3/3	3	3	39m
zookeeper	1/1	1	1	39m

Figure 4: Kubernetes deployment

```
krishnamohan_koyya@instance-1:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
kafka-588777fff6-86s24	1/1	Running	0	39m
mysqldb-8454bb5bdd-ph2xs	1/1	Running	0	39m
ums-add-service-7cb4dc9f9b-72xtq	1/1	Running	0	39m
ums-add-service-7cb4dc9f9b-cjx22	1/1	Running	0	39m
ums-add-service-7cb4dc9f9b-thgkn	1/1	Running	0	39m
zookeeper-6bb6f5f5-sl5mq	1/1	Running	0	39m

Figure 5: Pods

show the number of pods of each deployment that is running:

```
$ kubectl get deployments
```

The output looks like Figure 4.

And, finally, to see the real pods that run the containers, use the following command:

```
$ kubectl get pods
```

Figure 5 shows that there are three pods running for *AddService*, and one pod running for Zookeeper, Kafka and MySQL each.

Since the *AddService* is exposed with an *external-ip*, it can be accessed using the following command:

```
$ curl -X POST -H 'Content-Type: application/json'
-i http://<service-external-ip>:8080/user --data
'{"name": "Krishna Mohan", "phone": 9731423166}'
```

Why is this reference architecture?

Irrespective of the nature of the application, number of microservices, platforms on which they are developed, services like databases, brokers, etc, the overall architecture remains the same like what has been described here.

The development architecture focuses on service decomposition, platform selection, framework selection, design of API, repositories, etc. This part was addressed using our understanding of domain driven design, object-oriented patterns, frameworks like SpringBoot, Flask, Express, etc.

The deployment architecture focuses on number of machines, nodes, replica-sets, pods, address-mechanisms, volumes, etc. This part is addressed using our understanding of container technology and Kubernetes. We will dwell into design patterns associated exclusively with microservices like gateways, circuit breakers, registries, etc, in future. The good thing is that Kubernetes and other such tools implement many of these patterns out-of-the-box.

Before going that far, we will develop the *FindService*, *SearchService* and *JournalService* on Python and Node platforms in the next parts of this series of articles so that we take UMS to a conclusion.. 

 By: Krishna Mohan Koyya

The author is the founder and the principal technology consultant at Glarimy Technology Services, Bengaluru. He has delivered more than 500 programs to corporate clients on design and architecture in the last 12 years. Prior to that, he held various positions in the industry and academia for a decade.